

Departamento de Engenharia Eletrotécnica e de Computadores

Conversores de Sinais
2025/2026 - 2º Semestre
Relatório

Identificação

Turno: P2

Data: 20/05/2026

Docente: Professor Nuno Paulino

	<i>Nome</i>	<i>Número</i>
Aluno:	Miguel Pinho	7 3 3 1 9
Aluno:	Delmont Maria	7 5 2 1 8
Aluno:	Rodrigo Lapa	6 6 0 1 2

Índice

1 Introdução

A crescente necessidade por sistemas *wireless* de banda larga e de baixo consumo energético, e a migração dos nós IoT para tecnologias CMOS cada vez mais avançadas (como as de 90 ou até 55 nm) tem chamado mais a atenção da indústria eletrónica por investimento na área de pesquisa e desenvolvimento. Estes nós contam com blocos de lógica digital que é rápida e barata, mas a bateria limitada e tempo de trabalho exigido jogam contra os blocos analógicos. É, assim, interessante investir em conversores analógico digital (ADC) de consumo e área reduzidos. É assim que surge a abordagem por um ADC de registador de aproximações sucessivas (SAR ADC), por serem maioritariamente digitais e compostos por elementos passivos (condensadores), o que faz com que sejam energeticamente eficientes.

O seu principal problema, ou o problema com a sua variante mais tradicional, deve-se ao facto de a área aumentar exponencialmente com a resolução do ADC. Um *array* binário de condensadores de 12 bits precisaria, por exemplo, de $2^{12} C_{unit}$, o que aumenta não só a sua dimensão mas também o seu custo de produção.

O objetivo deste trabalho consiste no desenvolvimento de um modelo de alto nível de um SAR ADC de 12 bits, baseado na abordagem proposta por [?], que contempla a utilização de uma arquitetura dividida (*split array*) com recurso a um condensador de ponte (C_B), que reduz enormemente o número de condensadores. Pretende-se também efetuar uma análise de sensibilidade a erros, como o de *offset* e *mismatch*, ou seja, desvio dos valores unitários para os condensadores, com recurso a simulações via Monte Carlo.

2 Modelação do SAR ADC de 12 bits

Desenvolver um modelo de alto nível para um SAR ADC, quanto mais o que consta no *paper* de [?], envolve, antes de qualquer simulação, a compreensão da sua estrutura e de uma análise matemática para facilitar a análise.

2.1 Arquitetura do conversor e redistribuição de carga

Ao contrário das abordagens mais convencionais do SAR ADC, que se servem de um *array* de condensadores binário (CDAC), este utiliza uma arquitetura dividida (*split*-CDAC) em duas partes por um condensador de ponte (C_B).

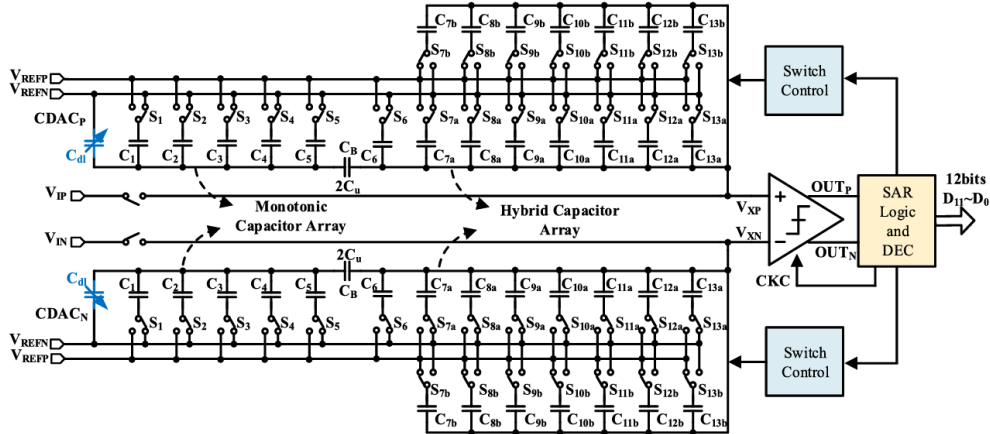


Figura 1: Esquemático do SAR ADC de 12 bits do *paper* e a arquitetura *split*-CDAC.

Assim, como vemos na figura ??, o *sub-array* monotónico, com os condensadores $[C_1 - C_5]$, lida com os bits menos significativos (LSBs) e inclui C_{dl} , que é um condensador programável para fazer ajustes e garantir a linearidade. O *sub-array* "híbrido" (que não o é inteiramente por causa do C_6) e que contempla os condensadores $[C_6 - C_{13a/b}]$ refere-se aos bits mais significativos (MSBs). É importante notar que, no exemplo do artigo, as chaves com as notações de letras (a e b, especificamente) por uma questão de redundância: existem mais passos de comparação que os 12 bits finais, e essa redundância permite absorver erros de *settling time* do DAC para uma conversão mais rápida e é ultimamente resolvida graças ao bloco DEC.

A vantagem desta abordagem consiste na redução de área e consumo energético. Se considerarmos um CDAC binário, temos que para uma resolução de N bits, seriam necessários 2^{N-1} condensadores. Com *split*-CDAC, o número de condensadores cai para $2^{n_{MSB}} + 2^{n_{LSB}}$, o que significa que é necessário menos energia para os *switches* de acordo com a fase, sem contar que menos condensadores implica área menor. A desvantagem, porém, é que pela utilização do C_B , as capacitâncias parasíticas propagam-se e provocam erros de ganho.

A operação para a redistribuição de carga baseia-se na lei da conservação de carga, e a comparação de valores começa logo na fase de *sampling* (Φ_S), em que é tomada a primeira decisão sobre se $V_{XP} > V_{xN}$. A análise, tal como a arquitetura, seguirá uma estrutura dividida, mas aqui em três partes, no que seria um 5-1-5: os primeiros cinco bits são reservados ao *sub-array* MSB, o C_6 (que está do lado dos MSBs, mas tem uma aparência mais monotónica que híbrida) e por fim o *sub-array* LSB.

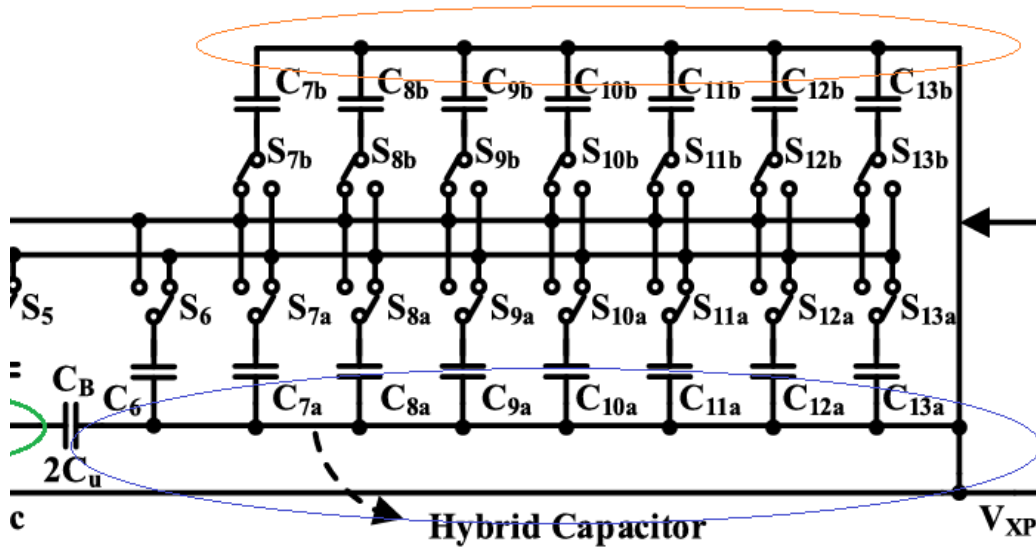


Figura 2: Análise do *sub-array* MSB, com os nós destacados.

Partimos assim para a análise do *array* MSB, e particularmente ao nó genérico V_X , já que a adaptação dos cálculos quer para V_{XN} como V_{XP} mostra-se trivial pela simetria do CDAC. Seja $C_{MSB} = C_{MSBa} + C_{MSBb}$, e $C_{MSBa,b} = \sum_{i=7}^{11} C_{i,a,b}$. Para auxílio dos cálculos, o nó destacado a verde vai ser denominado V_{aux} , que na fase Φ_S é igual a 0; o nó associado à cor azul refere-se ao lado das chaves "a", e o nó a laranja associado às chaves "b". A partir desta expressão, podemos assumir que a carga associada ao nó V_{XP} e à parte MSB do *sub-array* na fase de *sampling* pode ser dada por:

$$\begin{aligned} Q_{MSB}^{\Phi_S} &= C_{MSBa} \left(V_X^{\Phi_S} - V_{REFN} \right) + C_{MSBb} \left(V_X^{\Phi_S} - V_{REFP} \right) \\ &\quad + C_6 \left(V_X^{\Phi_S} - V_{REFN} \right) \\ &\quad + C_B \left(V_X^{\Phi_S} - V_{aux}^{\Phi_S} \right) \\ &= V_{IP,N} (C_{MSB} + C_B) - V_{REFN} (C_{MSBa} + C_6) - V_{REFP} C_{MSBb} \end{aligned} \quad (1)$$

ao passo que a carga na fase a seguir, Φ_n , que é genérica, para que se aplique a todos os casos, é dada por

$$\begin{aligned} Q_{MSB}^{\Phi_n} &= C_{MSBa} \left(V_X^{\Phi_n} - SW \cdot V_{REFP} - \overline{SW} \cdot V_{REFN} \right) + C_{MSBb} \left(V_X^{\Phi_n} - SW \cdot V_{REFN} - \overline{SW} \cdot V_{REFP} \right) \\ &\quad + C_6 \left(V_X^{\Phi_n} - SW \cdot V_{REFP} - \overline{SW} \cdot V_{REFN} \right) \\ &\quad + C_B \left(V_X^{\Phi_n} - V_{aux}^{\Phi_n} \right) \\ &= (C_{MSBa} + C_6) \left(V_X^{\Phi_n} - SW \cdot V_{REFP} - \overline{SW} \cdot V_{REFN} \right) \\ &\quad + C_{MSBb} \left(V_X^{\Phi_n} - SW \cdot V_{REFN} - \overline{SW} \cdot V_{REFP} \right) + C_B \left(V_X^{\Phi_n} - V_{aux}^{\Phi_n} \right) \end{aligned} \quad (2)$$

É importante notar que a generalização implica a utilização do SW , que representa o *switch control* associado ao bit em análise. Pela lei da conservação da carga, igualamos as equações ?? a ?? para obter

$$\begin{aligned} V_{IP,N} (C_{MSB} + C_B) &= (C_{MSBa} + C_6) \left(V_X^{\Phi_n} - SW \cdot V_{REFP} - \overline{SW} \cdot V_{REFN} + V_{REFN} \right) \\ &\quad + C_{MSBb} \left(V_X^{\Phi_n} - SW \cdot V_{REFN} - \overline{SW} \cdot V_{REFP} + V_{REFP} \right) + C_B \left(V_X^{\Phi_n} - V_{aux}^{\Phi_n} \right) \end{aligned} \quad (3)$$

ou, simplificando $\overline{SW} = 1 - SW$, temos

$$V_{IP,N} (C_{MSB} + C_B) = (C_{MSB_a} + C_6) (V_X^{\Phi_n} - SW \cdot (V_{REFP} - V_{REFN})) + C_{MSB_b} (V_X^{\Phi_n} - SW \cdot (V_{REFN} - V_{REFP})) + C_B (V_X^{\Phi_n} - V_{aux}^{\Phi_n}) \quad (4)$$

Analisando agora a porção LSB do *array*, deparamo-nos com um paradigma mais trivial, já que se trata de um *array* monotónico, e que não exige tanta abstracção como no caso do *sub-array* MSB.

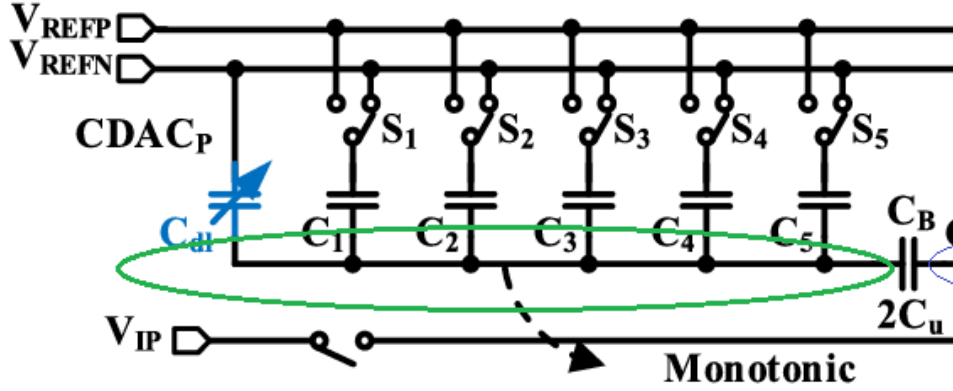


Figura 3: Análise do *sub-array* LSB, com os nós destacados.

Tal como no caso MSB, servimo-nos do mesmo nó genérico V_X , a mesma variável auxiliar V_{aux} , que na fase Φ_S é também igual a 0 e que está associada ao nó verde da imagem ?? e do facto de que o CDAC é simétrico, para permitir uma análise a apenas um dos lados.

Seja $C_{LSB} = \sum_{i=1}^5 C_i$, e $C_{LSB_{tot}} = C_{LSB} + C_{dl}$. Assim, podemos inferir que a carga associada ao nó na fase de *sampling* é dada por

$$Q_{LSB}^{\Phi_S} = C_{LSB_{tot}} (V_{aux}^{\Phi_S} - V_{REFN}) + C_B (V_{aux}^{\Phi_S} - V_X^{\Phi_S}) = - (V_{IP,N} C_B + V_{REFN} C_{LSB_{tot}}) \quad (5)$$

Analogamente, para a fase seguinte é dada por

$$Q_{LSB}^{\Phi_n} = C_{LSB} (V_{aux}^{\Phi_n} - SW \cdot V_{REFP}) + C_{dl} (V_{aux}^{\Phi_n} - V_{REFN}) + C_B (V_{aux}^{\Phi_n} - V_X^{\Phi_n}) \quad (6)$$

Pela lei da conservação da carga, igualamos as equações ?? a ?? e obtemos

$$- (V_{IP,N} C_B + V_{REFN} C_{LSB_{tot}}) = C_{LSB} (V_{aux}^{\Phi_n} - SW \cdot V_{REFP}) + C_{dl} (V_{aux}^{\Phi_n} - V_{REFN}) + C_B (V_{aux}^{\Phi_n} - V_X^{\Phi_n}) \quad (7)$$

o que nos permite isolar $V_{aux}^{\Phi_n}$, obtendo

$$V_{aux}^{\Phi_n} = \frac{C_{LSB} (SW \cdot V_{REFP} - V_{REFN}) + C_B (V_X^{\Phi_n} - V_{IP,N})}{C_{LSB_{tot}} + C_B} \quad (8)$$

Assim, o objetivo torna-se substituir o valor obtido na equação ?? na equação ?. Começamos por expandir e simplificar a equação ??:

$$V_{I_{P,N}}(C_{MSB} + C_B) = V_X^{\Phi_n}(C_{MSB_a} + C_6 + C_{MSB_b} + C_B) - SW(V_{REF_P} - V_{REF_N})(C_{MSB_a} + C_6 - C_{MSB_b}) - C_B V_{aux}^{\Phi_n}$$

Em seguida, para facilitar a substituição na parte final, multiplicamos à parte por C_B , obtendo

$$C_B V_{aux}^{\Phi_n} = \frac{C_B C_{LSB}}{C_{LSB_{tot}} + C_B} (SW \cdot V_{REF_P} - V_{REF_N}) + \frac{C_B^2}{C_{LSB_{tot}} + C_B} V_X^{\Phi_n} - \frac{C_B^2}{C_{LSB_{tot}} + C_B} V_{I_{P,N}}$$

Substituindo finalmente, ficamos com

$$\begin{aligned} V_{I_{P,N}}(C_{MSB} + C_B) &= V_X^{\Phi_n}(C_{MSB_a} + C_6 + C_{MSB_b} + C_B) \\ &\quad - SW(V_{REF_P} - V_{REF_N})(C_{MSB_a} + C_6 - C_{MSB_b}) \\ &\quad - \frac{C_B C_{LSB}}{C_{LSB_{tot}} + C_B} (SW \cdot V_{REF_P} - V_{REF_N}) \\ &\quad - \frac{C_B^2}{C_{LSB_{tot}} + C_B} V_X^{\Phi_n} \\ &\quad + \frac{C_B^2}{C_{LSB_{tot}} + C_B} V_{I_{P,N}} \leftrightarrow \\ \leftrightarrow V_{I_{P,N}} \left(C_{MSB} + C_B - \frac{C_B^2}{C_{LSB_{tot}} + C_B} \right) &= V_X^{\Phi_n} \left(C_{MSB_a} + C_6 + C_{MSB_b} + C_B - \frac{C_B^2}{C_{LSB_{tot}} + C_B} \right) \\ &\quad - SW(V_{REF_P} - V_{REF_N})(C_{MSB_a} + C_6 - C_{MSB_b}) \\ &\quad - \frac{C_B C_{LSB}}{C_{LSB_{tot}} + C_B} (SW \cdot V_{REF_P} - V_{REF_N}) \leftrightarrow \\ \leftrightarrow V_X^{\Phi_n} \left(C_{MSB_a} + C_6 + C_{MSB_b} + \frac{C_B C_{LSB_{tot}}}{C_{LSB_{tot}} + C_B} \right) &= V_{I_{P,N}} \left(C_{MSB} + \frac{C_B C_{LSB_{tot}}}{C_{LSB_{tot}} + C_B} \right) \\ &\quad + SW(V_{REF_P} - V_{REF_N})(C_{MSB_a} + C_6 - C_{MSB_b}) \\ &\quad + \frac{C_B C_{LSB}}{C_{LSB_{tot}} + C_B} (SW \cdot V_{REF_P} - V_{REF_N}) \end{aligned}$$

Considerando que $C_{eq} = C_{MSB_a} + C_6 + C_{MSB_b} + \frac{C_B C_{LSB_{tot}}}{C_{LSB_{tot}} + C_B}$, temos que

$$V_X^{\Phi_n} = \frac{V_{I_{P,N}} C_{eq} + SW(V_{REF_P} - V_{REF_N})(C_{MSB_a} + C_6 - C_{MSB_b}) + \frac{C_B C_{LSB}}{C_{LSB_{tot}} + C_B} (SW \cdot V_{REF_P} - V_{REF_N})}{C_{eq}} \quad (9)$$

Simplificando $V_{I_{P,N}} = V_{CM} \pm \frac{\Delta V_i}{2}$ (mais se for o nó P, menos se for o nó N) e $\alpha = \frac{C_B}{C_{LSB_{tot}} + C_B}$ como fator de atenuação devido à presença do C_B , temos

$$V_X^{\Phi_n} = V_{CM} \pm \frac{\Delta V_i}{2} + \frac{1}{C_{eq}} (SW(V_{REF_P} - V_{REF_N})(C_{MSB_a} + C_6 - C_{MSB_b}) + \alpha(SW \cdot V_{REF_P} - V_{REF_N})C_{LSB}) \quad (10)$$

2.2 Modelação comportamental sem e com erros

Para validar o comportamento do SAR ADC, foi desenvolvido um modelo comportamental em Python. O modelo implementa, de forma discreta no tempo, o ciclo de amostragem e rotina de aproximações sucessivas descrita no *paper*.

A arquitetura do código segue três fases distintas:

1. Definição da topologia do CDAC, incluindo os pesos binários dos *sub-arrays* MSB e LSB, o C_B , o C_6 e as capacitâncias parasíticas C_{pb} , C_{pl} e C_{pm} .
2. Introdução de erros através de distribuições gaussianas para o *mismatch* dos condensadores e o offset do comparador
3. Extração de métricas de desempenho, como a INL, DNL, SNDR e ENOB

O *matching* entre condensadores é um dos fatores limitantes da resolução de ADCs capacitivos. No modelo, assume-se que o desvio padrão relativo da capacitância segue a lei de Pelgrom:

$$\sigma\left(\frac{\Delta C}{C}\right) = \frac{A_C}{\sqrt{W \cdot L}}$$

Na prática, para um determinado desvio padrão σ_C , o valor real de cada condensador é dada por

$$C_{real} = C_{ideal} + C_{ideal} \cdot \sigma_c \cdot N(0,1)$$

onde $N(0,1)$ é uma variável aleatória com distribuição normal padrão.

```
# Função para calcular os valores dos condensadores considerando os erros
def calcular_condensadores(SC, Cu, Cpb, Cpl, Cpm, Nlsb):
    valores_CiMSB = []
    for m in multiplicadores_split_MSB:
        unidades_a = (Cu/2) + (SC * np.random.randn(m))
        unidades_b = (Cu/2) + (SC * np.random.randn(m))
        valores_CiMSB.append(np.sum(unidades_a) + np.sum(unidades_b))

    valores_CiLSB = []
    for m in multiplicadores_split_LSB:
        unidades_lsb = Cu + (SC * np.random.randn(m))
        valores_CiLSB.append(np.sum(unidades_lsb))

    C6 = 1.0 * Cu

    # Para valores de Cb e Cdl:
    Cb = (2**Nlsb / ((2**Nlsb) - 1)) * Cu + Cpb
    #Cb = 2*Cu
    Cdl = (2**Nlsb - 1)*(Cb-Cu) - Cpl

    CtotalMSB = sum(valores_CiMSB) + Cpm
    CtotalLSB = sum(valores_CiLSB) + Cdl + Cpl

    C_ramo_LSB = (Cb * CtotalLSB) / (Cb + CtotalLSB)
    Ctot = CtotalMSB + C6 + C_ramo_LSB

    return valores_CiMSB, valores_CiLSB, C6, Ctot, Cb, CtotalLSB
```

Figura 4: Cálculo dos valores dos condensadores

Conforme visto na figura ??, todos os condensadores contam com parâmetros não-ideais, mas que para a validação do modelo sem erros são postos a zero, como se não estivessem lá.

É importante analisar a influência do condensador C_B . Pela fórmula teórica,

$$C_B = \frac{2^{N_{LSB}}}{2^{N_{LSB}} - 1} \cdot C_u = \frac{2^5}{2^5 - 1} \cdot C_u$$

logo, considerando que N_{LSB} , para a abordagem 5-1-5 que escolhemos é 5, o tamanho ideal seria $\frac{32}{31} \cdot C_u$.

Esta configuração assegura que a capacitância equivalente vista da matriz MSB mantém uma razão puramente binária. O peso do bit menos significativo do *sub-array* MSB é exatamente igual a $1LSB_{MSB} = 2^5 \cdot LSB_{total} = 32LSB$. A transição entre o maior código do *sub-array* LSB (00000 11111) e o primeiro código MSB (00001 00000) ocorre sem qualquer descontinuidade lógica, com uma INL nula. Porém, em termos de layout físico, fabricar um condensador com uma razão não-inteira é extremamente difícil, a um ponto em que rouba a viabilidade.

Por isso, muitas vezes é preferível a abordagem por um valor inteiro ou aproximado por potências como é $C_B = 2 \cdot C_u$. Por ser um múltiplo da unidade básica, o condensador C_B pode ser desenhado de forma a emparelhar perfeitamente a geometria aos restantes condensadores do *array*, o que reduziria os erros aleatórios de fabrico. Por outro lado, alterar o valor de C_B para $2 \cdot C_u$ faz com que o fator de atenuação α da equação ?? altere, e o ganho do *sub-array* MSB deixa de estar alinhado ao LSB, o que geraria erros de ganho fracionário sistemáticos.

Em seguida, a função `sar_converter()` representa o processo de conversão de um dado valor de tensão diferencial à entrada através da comparação das tensões V_{XP} e V_{XN} . Durante a fase de amostragem, as placas superiores dos condensadores são ligadas à tensão de modo comum $V_{CM} = \frac{V_{REF}}{2}$, ao passo que as placas inferiores adquirem os sinais diferenciais. Em seguida, o algoritmo percorre sequencialmente o *array*, e para cada bit j , o procedimento passa por inspecionar o bit anterior e comutar o condensador correspondente para V_{REFP} (Vref) ou V_{REFN} , que para o nosso caso era GND. A tensão dos nós V_{XP} e V_{XN} é atualizada com base na equação ?. Em seguida, é tomada uma decisão: se $V_{XP} - V_{XN} + V_{off} > 0$, o bit $j = 1$. Caso contrário, é 0. Importante notar que o fator de atenuação α da equação ?? é o mesmo usado no código, considerando também a componente não-ideal para refletir o efeito do condensador *bridge*.

Posteriormente, para obter a função de transferência estática `dout(Vin)`, o código percorre uma rampa de entrada de -Vref a Vref com uma resolução de 20 pontos por LSB, em que o vetor `codigos` armazena o resultado decimal da conversão para cada ponto da rampa. A partir da função transferência, determinamos as tensões de transição $V_{trans}[k]$, o LSB real, DNL e INL.

Para avaliar o desempenho em regime dinâmico, o modelo aplica um sinal sinusoidal de entrada com frequência f_{in} escolhida de forma a ser incoerente com a frequência de amostragem f_s .

$$f_{in} = \frac{M}{N_{FFT}} \cdot f_s$$

em que M é um número primo. Assumimos $N_{FFT} = 2^{N_{bits}+2}$ e $M = 101$. O sinal amostrado é em seguida multiplicado por uma janela de Blackman para reduzir o *spectral leakage*:

$$w[n] = 0,42 - 0,5 \cos\left(\frac{2\pi n}{N-1}\right) + 0,08 \cos\left(\frac{4\pi n}{N-1}\right)$$

A FFT permite-nos, em seguida, calcular a potência do sinal P_s , potência das harmónicas P_d , potência do ruído ($P_n = P_{total} - P_s - P_d$), a relação sinal-ruído e distorção SNDR e o número efetivo de bits ENOB.

A função final do código executa uma análise de Monte Carlo com 1000 amostras, para diferentes valores de desvio $\sigma \frac{\Delta C}{C}$, em que para cada chip e cada σ são gerados novos valores de condensadores com erros aleatórios, que simula a variabilidade no processo de fabrico.

2.2.1 Validação do modelo

Para validar o modelo, avaliamos as métricas de desempenho citadas na subsecção ??.

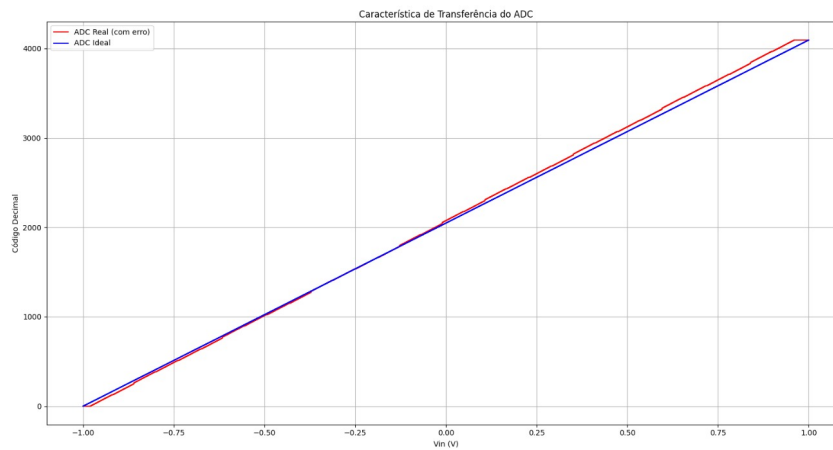
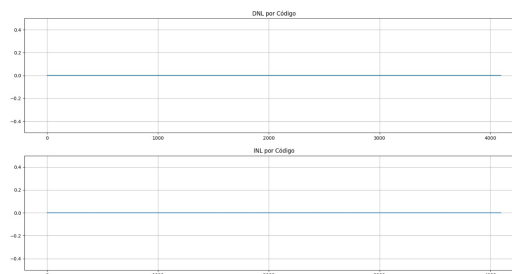
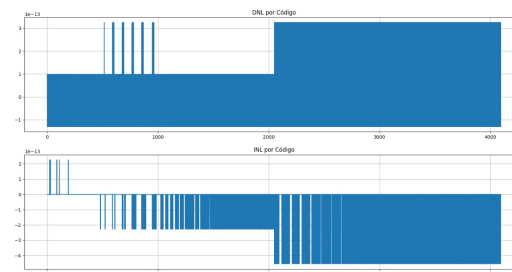


Figura 5: Função transferência ideal e função transferência de um ADC real.



(a) INL e DNL com parâmetros não-ideais a 0.



(b) INL e DNL com parâmetros não-ideais diferentes de 0.

Figura 6: Análise da INL e DNL

Em condições absolutamente ideais, todos os condensadores possuem os valores nominais exatos ($C_j = 2^k \cdot C_u$). A função transferência apresenta degraus de quantização perfeitamente uniformes com largura igual a 1 LSB. A DNL e a INL em todos os códigos são zero, o que significa que tanto não há variação na largura dos intervalos de transição como o centro de cada degrau real coincide perfeitamente com a linha reta ideal que une a origem ao fundo da escala do conversor.

Quando introduzimos o erro estatístico de *mismatch* capacitivo baseado no modelo de Pelgrom, a geometria real dos condensadores sofre desvios aleatórios, o que altera os pesos binários ideais da equação de redistribuição da carga.

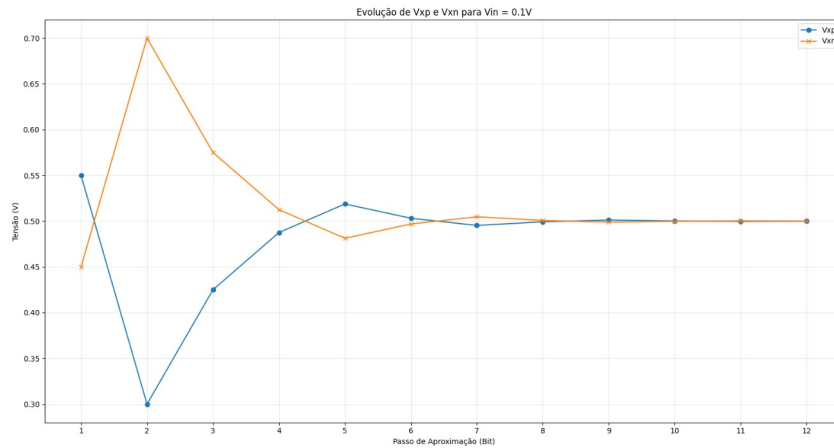


Figura 7: Evolução do nó V_{XP} e do nó V_{XN} .

Pela natureza do método de aproximações sucessivas, a diferença $\Delta V = V_{XP} - V_{XN}$ tende para zero a cada iteração, uma vez que o algoritmo minimiza o erro diferencial. As tensões absolutas V_{XP} e V_{XN} podem tender cada uma para valores díspares do V_{CM} , na presença de *mismatch* ou *offset*, ou até pelo facto de a arquitetura monotónica não ser capaz de manter a tensão de modo comum fixa como a híbrida, mas em casos ideais a diferença anular-se-ia e as tensões dos nós convergiriam para V_{CM} .

2.3 Análise Monte Carlo

Tendo em conta a variabilidade da capacitância dos condensadores no processo de fabrico, é necessário ter uma noção do comportamento do ADC perante esta possibilidade. Um dos métodos usados para a previsão do comportamento é a análise Monte Carlo. Nesta análise verificámos o comportamento do ADC perante a variação dos condensadores com um desvio padrão de 0% a 0,12%. Foram simulados 1000 casos para cada desvio padrão, com 10 desvio padrão diferentes, inseridos dentro do intervalo referido. Em cada caso foi retirado o valor da SNDR, que será registado de modo a obter uma distribuição da SNDR para cada desvio padrão. Para cada desvio padrão será calculada a SNDR máxima, média e mínima. Esta análise pode ser vista na figura ??.

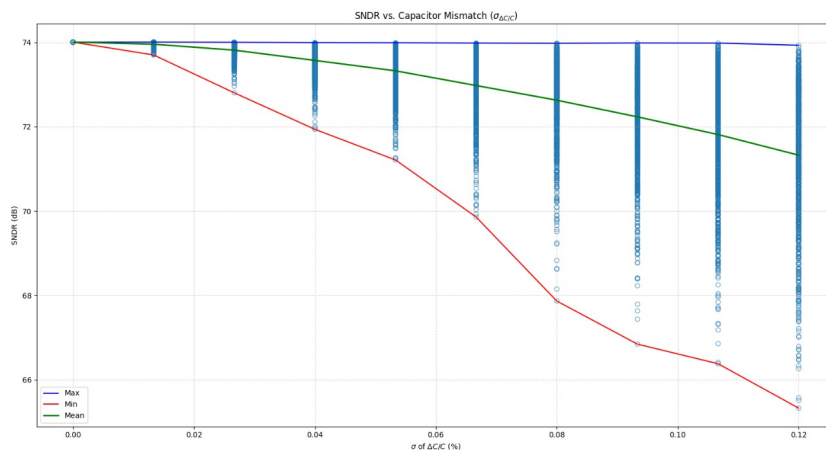


Figura 8: Mismatch entre os condensadores e influência na SNDR.

Como é expetável, a SNDR média diminui à medida que o desvio padrão aumenta, tendo uma queda de 2dB para um desvio padrão de 0,10%. No entanto, esta queda é pequena em comparação com o pior caso neste desvio, que ronda os 7dB. A distância observada entre a SNDR média e a SNDR mínima indica que a probabilidade de uma queda acentuada da SNDR é relativamente baixa.

2.4 Estimativa de área e dissipação de potência

Para poder estimar a área e a dissipação de potência para um ENOB de 10,5 bits, servimo-nos da relação entre o SNDR e o ENOB dada por $ENOB = \frac{SNDR - 1,76}{6,02}$. É, assim, apenas necessário encontrar o *mismatch* associado ao ENOB solicitado.

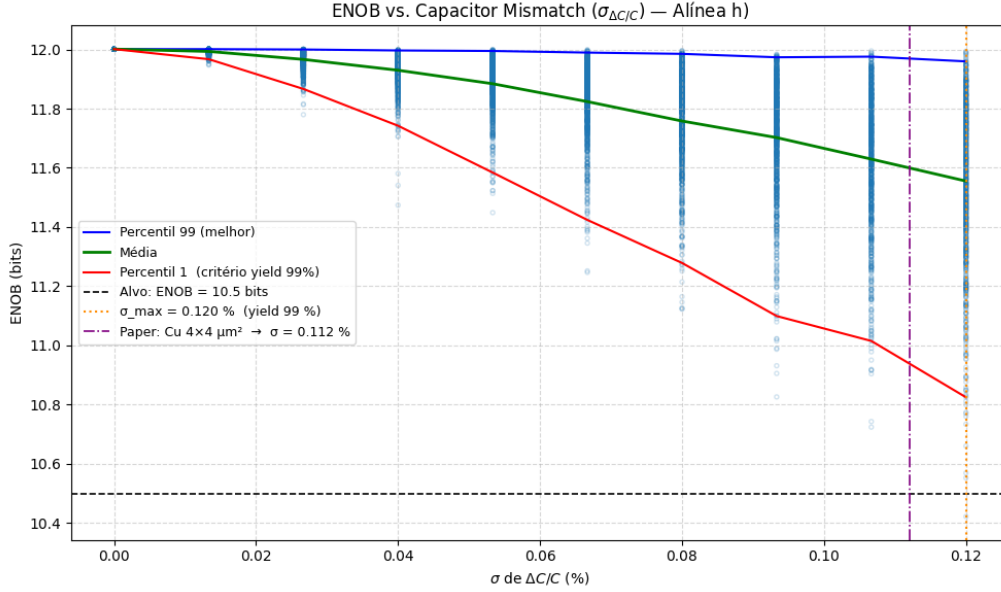


Figura 9: Mismatch entre os condensadores e relação com ENOB.

Conforme é possível observar na figura ??, o σ_{max} é de aproximadamente 0,120%. Utilizando a equação da reta da figura 4 do *paper*, obtemos o valor da área, dado por

$$\frac{0,120 - 0,01}{0,408} = \frac{1}{\sqrt{Area}} \quad (11)$$

Desenvolvendo, tem-se que a área é igual a $13,75 \mu m^2$. Como a referência seria de 6,5fF para um condensador de $16 \mu m^2$, a capacitância do condensador unitário seria dada por

$$C_u = 13,75 \cdot 0,406 fF / \mu m^2 = 5,58 fF$$

Para a potência dissipada, consideramos o $C_{total} = 129 \cdot C_u \cdot 2 = 1440 fF$, considerando o dobro da capacitância total associada a um CDAC *single-ended*. Pela fórmula

$$P_d \approx f_s * C_{tot} * V_{ref}^2 \quad (12)$$

Com $V_{REF} = 1V$, tem-se que $P_d \approx 100 MHz \cdot 1440 fF \cdot 1 \approx 144 \mu W$.

3 Conclusão

Este projeto permitiu o desenvolvimento e a validação de um modelo comportamental de alto nível em Python para um SAR ADC de 12 bits a 100 MS/s baseado numa arquitetura *split*-CDAC de topologia 5-1-5, tendo por referência o trabalho desenvolvido em [?]. A dedução rigorosa das expressões analíticas de redistribuição de carga com base na lei da conservação da carga revelou-se fundamental para mapear com precisão a evolução das tensões nos nós internos V_{XP} e V_{XN} ao longo do processo de aproximações sucessivas.

A análise detalhada da matriz de condensadores dividida evidenciou o compromisso crítico entre a eficiência de área/energia e a sensibilidade a não-idealidades. Demonstrou-se que, embora o *split*-CDAC reduza exponencialmente o número total de condensadores necessários face à abordagem binária convencional — minimizando o consumo dinâmico dos *switches* —, introduz uma vulnerabilidade acrescida devido ao condensador de ponte (C_B). A comparação entre o valor ideal e fracionário $C_B = \frac{32}{31} C_u$ (que assegura linearidade perfeita e INL nula num modelo ideal) e o valor inteiro $C_B = 2 C_u$ expôs a dicotomia clássica entre a viabilidade física de *layout* microelectrónico e o aparecimento de erros de ganho sistemáticos entre os *sub-arrays* MSB e LSB.

Através da simulação de Monte Carlo com 1000 iterações, quantificou-se o impacto do *mismatch* capacitivo estático (baseado na lei de Pelgrom) e do *offset* do comparador nas métricas de linearidade (INL e DNL) e desempenho dinâmico (SNDR e ENOB). Observou-se uma degradação expectável e progressiva da SNDR média com o aumento do desvio padrão do erro, sublinhando que variações extremas (pior caso) podem comprometer severamente a integridade do sinal, embora apresentem baixa probabilidade de ocorrência. Ainda assim, deve-se ter em conta que o SAR ADC de [?], ao contrário do conversor em análise, conta com um bloco de correção digital de erros, o que permite obter resultados melhores que os obtidos.

Por fim, os requisitos estipulados para uma linearidade de target de 10, 5 bits de ENOB permitiram extrair as restrições geométricas do circuito. Identificou-se que um desvio padrão máximo admissível de $\sigma_{max} \approx 0,120\%$ impõe uma área mínima de matriz de $13,75 \mu\text{m}^2$, correspondendo a uma capacitância unitária C_u de 5,58 fF. Sob estas condições, a estimativa da potência dissipada pelo CDAC diferencial fixou-se em aproximadamente $144 \mu\text{W}$. Conclui-se, assim, que o modelo de alto nível desenvolvido constitui uma ferramenta preditiva robusta e eficaz para o dimensionamento preliminar de conversores analógico-digitais SAR, validando os benefícios de baixo consumo e alta velocidade da arquitetura estudada.

Referências

- [Li et al., 2021] Li, M., Yao, Y., Hu, B., Wei, J., Chen, Y., Ma, S., Ye, F., and Ren, J. (2021). A 6.94-fj/conversion-step 12-bit 100-ms/s asynchronous sar adc exploiting split-cdac in 65-nm cmos. *IEEE Access*, 9:77545–77554.

4 Anexos

Código para simulação do ADC

```
import random
import matplotlib.pyplot as plt
import numpy as np
from scipy.interpolate import interp1d

# Parametros do ADC
Nbits = 12
Vref = 1
Vlsb = (2 * Vref) / (2**Nbits)
codigos = []
codigos_real = []

multiplicadores_split_MSB = [32, 16, 8, 4, 2]
multiplicadores_split_C6 = [1]
multiplicadores_split_LSB = [16, 8, 4, 2, 1]

Nmsb = len(multiplicadores_split_MSB)
LC6 = len(multiplicadores_split_C6)
Nlsb = len(multiplicadores_split_LSB)

# -----
# Funcao para gerar numeros aleatorios
def randn():
    return random.gauss(0, 1)

# Funcao para calcular os valores dos condensadores considerando os erros
def calcular_condensadores(SC, Cu, Cpb, Cpl, Cpm, Nlsb):
    valores_CiMSB = []
    for m in multiplicadores_split_MSB:
        unidades_a = (Cu/2) + (SC * np.random.randn(m))
        unidades_b = (Cu/2) + (SC * np.random.randn(m))
        valores_CiMSB.append(np.sum(unidades_a) + np.sum(unidades_b))

    valores_CiLSB = []
    for m in multiplicadores_split_LSB:
        unidades_lsb = Cu + (SC * np.random.randn(m))
        valores_CiLSB.append(np.sum(unidades_lsb))

    C6 = 1.0 * Cu

    # Para valores de Cb e Cdl:
    Cb = (2**Nlsb / ((2**Nlsb) - 1)) * Cu + Cpb
    #Cb = 2*Cu
    Cdl = (2**Nlsb - 1)*(Cb-Cu) - Cpl

    CtotalMSB = sum(valores_CiMSB) + Cpm
    CtotalLSB = sum(valores_CiLSB) + Cdl + Cpl

    C_amo_LSB = (Cb * CtotalLSB) / (Cb + CtotalLSB)
    Ctot = CtotalMSB + C6 + C_amo_LSB

    return valores_CiMSB, valores_CiLSB, C6, Ctot, Cb, CtotalLSB
```

```
# Funcao para simular o processo de conversao SAR
def sar_converter(Vxp, Vxn, Nmsb, Nlsb, valores_CiMSB, valores_CiLSB, C6, Ctot, Vref, Cb,
    bit = []
    vxp_hist = [Vxp]
    vxn_hist = [Vxn]

    # Conversao inicial
    if (Vxp-Vxn+v_off) < 0:
        bit.append(0)
    else:
        bit.append(1)

    # Conversao dos bits MSB
    for j in range(1, Nmsb+1):
        if bit[j-1] == 1:
            Vxp -= (valores_CiMSB[j-1] / Ctot) * Vref/2
            Vxn += (valores_CiMSB[j-1] / Ctot) * Vref/2
            if Vxp-Vxn+v_off > 0:
                bit.append(1)
            else:
                bit.append(0)

        elif bit[j-1] == 0:
            Vxp += (valores_CiMSB[j-1] / Ctot) * Vref/2
            Vxn -= (valores_CiMSB[j-1] / Ctot) * Vref/2

            if Vxp-Vxn+v_off > 0:
                bit.append(1)
            else:
                bit.append(0)

        vxp_hist.append(Vxp)
        vxn_hist.append(Vxn)

    # Conversao do bit C6
    if bit[5] == 1:
        Vxp -= (C6 / Ctot) * Vref/2
        Vxn += (C6 / Ctot) * Vref/2

        if Vxp-Vxn+v_off > 0:
            bit.append(1)
        else:
            bit.append(0)

    elif bit[5] == 0:
        Vxp += (C6 / Ctot) * Vref/2
        Vxn -= (C6 / Ctot) * Vref/2

        if Vxp-Vxn+v_off > 0:
            bit.append(1)
        else:
            bit.append(0)

    vxp_hist.append(Vxp)
    vxn_hist.append(Vxn)

    atenuacao = Cb / (CtotalLSB + Cpl + Cb)

    # Conversao dos bits LSB
    for j in range(7, Nlsb+7):
        if bit[j-1] == 1:
```



```

Vxp -= (valores_CiLSB[j-7] / Ctot) * Vref/2 * atenuacao
Vxn += (valores_CiLSB[j-7] / Ctot) * Vref/2 * atenuacao

if Vxp-Vxn+v_off > 0:
    bit.append(1)
else:
    bit.append(0)

elif bit[j-1] == 0:
    Vxp += (valores_CiLSB[j-7] / Ctot) * Vref/2 * atenuacao
    Vxn -= (valores_CiLSB[j-7] / Ctot) * Vref/2 * atenuacao

    if Vxp-Vxn+v_off > 0:
        bit.append(1)
    else:
        bit.append(0)

vxp_hist.append(Vxp)
vxn_hist.append(Vxn)

return bit[:12], vxp_hist, vxn_hist

# Funcao para converter a lista de bits em codigo decimal
def converter_para_decimal(lista_bits):
    codigo_decimal = 0
    for b in lista_bits:
        codigo_decimal = codigo_decimal << 1
        codigo_decimal = codigo_decimal | b
    return codigo_decimal

# Funcao calcular DNL e INL
def calcular_dnl_inl(vin, codigos, nbits):
    codigo = np.array(codigos)
    vin = np.array(vin)

    indices_mudanca = np.where(np.diff(codigo) > 0)[0]
    v_transicoes = vin[indices_mudanca]
    codigos_apos = codigo[indices_mudanca + 1]

    v_zero = v_transicoes[0]
    v_max = v_transicoes[-1]
    num_degraus = codigos_apos[-1] - codigos_apos[0]

    vlsb_real = (v_max - v_zero) / num_degraus

    v_ideais = v_zero + (codigos_apos - codigos_apos[0]) * vlsb_real
    larguras_reais = np.diff(v_transicoes)

    inl = (v_transicoes - v_ideais) / vlsb_real
    dnl = (larguras_reais / vlsb_real) - 1

    max_dnl_chip = np.max(np.abs(dnl))
    max_inl_chip = np.max(np.abs(inl))

    # Comentar para ver o Monte Carlo, senao aparecem muitos graficos

    # Graficos
    # plt.figure(figsize=(10, 8))
    # plt.subplot(2,1,1)

```

```
# plt.plot(codigos_apos[: -1], dnl)
# plt.title("DNL porCodigo")
# #plt.ylim(-0.5, 0.5)
# plt.grid(True)

# plt.subplot(2,1,2)
# plt.plot(codigos_apos, inl)
# plt.title("INL porCodigo")
# #plt.ylim(-0.5, 0.5)
# plt.grid(True)
# plt.show()

return max_dnl_chip, max_inl_chip

# Funcao para calcular FFT e metricas de desempenho
def calcular_fft_e_metricas(codigos, Nbits, plot=True):
    N = len(codigos)

    janela = np.blackman(N)
    sinal_ac = (np.array(codigos) - np.mean(codigos)) * janela

    espetro = np.fft.fft(sinal_ac)
    mag = np.abs(espetro)[:N//2]
    psd = mag**2

    sinal = np.argmax(psd)
    p_s = np.sum(psd[max(0, sinal-2) : min(N//2, sinal+3)])
    p_d = 0

    h_max = int((N/2) / sinal) if sinal > 0 else 0

    for i in range(2, h_max + 1):
        h_idx_teorica = i * sinal
        fold = h_idx_teorica % N

        if fold >= N // 2:
            h_idx_real = N - fold
        else:
            h_idx_real = fold

        h_idx_final = int(h_idx_real)

        if h_idx_final > 2 and h_idx_final < (N // 2 - 3):
            p_d += np.sum(psd[h_idx_final-2 : h_idx_final+3])

    p_total = np.sum(psd)
    p_n = p_total - p_s - p_d
    p_n = max(p_n, 1e-20)

    snr = 10 * np.log10(p_s / (p_n + p_d))
    enob = (snr - 1.76) / 6.02

    mag_db = 20 * np.log10(mag / np.max(mag))
    freq_norm = np.linspace(0, 0.5, len(mag_db))

    # Comentar para ver o Monte Carlo, senao aparecem muitos graficos

    ## Grafico
    # if plot:
    #     # Tudo o que seja plt. deve estar indentado aqui dentro
```

```
# plt.figure(figsize=(10, 6))
# plt.plot(freq_norm, mag_db, color='blue', linewidth=0.7)
# plt.title(f"Dynamic Performance Analysis ({Nbits} bits)")
# plt.xlabel("Frequencia (MHz)")
# plt.ylabel("Magnitude (dB)")
# plt.ylim([-140, 5])
# plt.grid(True, which='both', linestyle='--', alpha=0.5)

# info_text = (f"Fin: {fin/1e6:.3f} MHz\n"
#              f"SNDR: {sندر:.2f} dB\n"
#              f"ENOB: {enob:.2f} bits ")
# plt.text(0.65, 0.95, info_text, transform=plt.gca().transAxes,
#          verticalalignment='top', bbox=dict(facecolor='white', alpha=0.7),
#          fontsize=10, family='monospace')
# plt.show()
# else:
#     plt.close('all')

return {"SNDR": sندر, "ENOB": enob}
```

```
# -----
# Simulacao do processo de conversao para cada valor de Vin
num_codigos = 2**Nbits
pontos_por_lsb = 20
amostras = num_codigos * pontos_por_lsb

Vin_inicial = np.linspace(-Vref, Vref, amostras)

Cu = 1
SC = 0
Soffset = 0
Cpb = 0
Cpl = 0
Cpm = 0

valores_CiMSB, valores_CiLSB, C6, Ctot, Cb, CtotalLSB = calcular_condensadores(SC, Cu, Cpb)

for i in range(amostras):
    Vin = Vin_inicial[i]
    Vip = (Vref/2) + (Vin/2)
    Vin = (Vref/2) - (Vin/2)
    Vxp = Vip
    Vxn = Vin
    res_sar, h_vxp, h_vxn = sar_converter(Vxp, Vxn, Nmsb, Nlsb, valores_CiMSB, valores_CiLSB)
    codigos.append(converter_para_decimal(res_sar))

# Plot da caracteristica de transferencia do ADC
plt.figure(figsize=(10,6))
plt.step(Vin_inicial, codigos)
plt.title("Caracteristica de Transferencia do ADC")
plt.xlabel("Vin (V)")
plt.ylabel("Codigo Decimal")
plt.grid(True)
plt.show()

# -----
# Simulacao do processo de conversao para cada valor de Vin mas com erro nos condensadores
SC_r = 0.03
Soffset_r = 0.01
Cpb_r = 1
```

```
Cpl_r = 1
Cpm_r = 1
```

```
valores_CiMSB, valores_CiLSB, C6, Ctot, Cb, CtotalLSB = calcular_condensadores(SC_r, Cu, C
```

```
for i in range(amostras):
    Vin = Vin_inicial[i]
    Vip = (Vref/2) + (Vin/2)
    Vin = (Vref/2) - (Vin/2)
    Vxp = Vip
    Vxn = Vin
    res_sar_real, h_vxp, h_vxn = sar_converter(Vxp, Vxn, Nmsb, Nlsb, valores_CiMSB, valores_
    codigos_real.append(converter_para_decimal(res_sar_real))
```

```
# Plot da caracteristica de transferencia do ADC
```

```
plt.figure(figsize=(10,6))
plt.step(Vin_inicial, codigos_real, color='red', label='ADC_Real(com_erro)', where='post')
plt.step(Vin_inicial, codigos, color='blue', label='ADC_Ideal', where='post')
plt.title("Caracteristica de Transferencia do ADC")
plt.xlabel("Vin(V)")
plt.ylabel("Codigo Decimal")
plt.legend()
plt.grid(True)
plt.show()
```

```
#
```

```
# Vxp e Vxn
```

```
Vin_exemplo = 0.1
Vip = (Vref/2) + (Vin_exemplo/2)
Vin = (Vref/2) - (Vin_exemplo/2)
Vxp = Vip
Vxn = Vin
```

```
res, h_vxp, h_vxn = sar_converter(Vip, Vin, Nmsb, Nlsb, valores_CiMSB, valores_CiLSB, C6,
```

```
plt.figure(figsize=(12, 6))
plt.plot(h_vxp, 'o-', label='Vxp')
plt.plot(h_vxn, 'x-', label='Vxn')
```

```
posicoes = range(len(h_vxp))
labels_bits = [str(i+1) for i in posicoes]
```

```
plt.title(f"Evolucao de Vxp e Vxn para Vin={Vin_exemplo}V")
plt.xlabel("Passo de Aproximacao (Bit)")
plt.ylabel("Tensao(V)")
plt.xticks(posicoes, labels=labels_bits)
plt.grid(True, alpha=0.3)
plt.legend()
plt.show()
```

```
#
```

```
# DNL e INL
```

```
calcular_dnl_inl(Vin_inicial, codigos, Nbits)
```

```
calcular_dnl_inl(Vin_inicial, codigos_real, Nbits)
```

```
#
```

```
# Calculo de FFT e metricas de desempenho
```

```

N_fft = 2*(Nbits + 2)
fs = 100e6
M = 101
fin = (fs / N_fft) * M
t = np.arange(N_fft) / fs
Vin_seno = (Vref - Vlsb) * np.sin(2 * np.pi * fin * t)
codigos_fft = []

valores_CiMSB, valores_CiLSB, C6, Ctot, Cb, CtotalLSB = calcular_condensadores(SC, Cu, Cpb, Cpl)

for v in Vin_seno:
    Vip = (Vref/2) + (v/2)
    Vin = (Vref/2) - (v/2)
    Vxp = Vip
    Vxn = Vin
    res_sar, h_vxp, h_vxn = sar_converter(Vxp, Vxn, Nmsb, Nlsb, valores_CiMSB, valores_CiLSB, C6, Ctot, Cb, CtotalLSB)
    codigos_fft.append(converter_para_decimal(res_sar))

calcular_fft_e_metricas(codigos_fft, Nbits)

# -----
# Monte Carlo
SC_mc = np.linspace(0, 0.0012, 10)
Numero_chips = 1000

sndr_results = np.zeros((len(SC_mc), Numero_chips))

for s_idx, sigma in enumerate(SC_mc):
    for i in range(Numero_chips):
        CiMSB, CiLSB, C6, Ctot, Cb, CtotalLSB = calcular_condensadores(sigma, Cu, Cpb, Cpl)

        codigos_seno = []
        for v in Vin_seno:
            Vp = (Vref/2 + v/2)
            Vn = (Vref/2 - v/2)
            res, h_vxp, h_vxn = sar_converter(Vp, Vn, Nmsb, Nlsb, CiMSB, CiLSB, C6, Ctot, Cb, CtotalLSB)
            codigos_seno.append(converter_para_decimal(res))

        metrics = calcular_fft_e_metricas(codigos_seno, Nbits)
        sndr_results[s_idx, i] = metrics["SNDR"]

sigma_pct = SC_mc * 100

# Calculo das estatisticas finais
sndr_max = np.max(sndr_results, axis=1)
sndr_min = np.min(sndr_results, axis=1)
sndr_mean = np.mean(sndr_results, axis=1)

plt.figure(figsize=(12, 7))

# 1. Desenha a nuvem de pontos azuis
for s_idx, s_val in enumerate(sigma_pct):
    plt.scatter(np.ones(Numero_chips) * s_val, sndr_results[s_idx, :],
                facecolors='none', edgecolors='tab:blue', alpha=0.3, s=10)

# 2. Desenha as linhas de tendencia
plt.plot(sigma_pct, sndr_max, color='blue', label='Max (Best Case)', linewidth=1.5)
plt.plot(sigma_pct, sndr_min, color='red', label='Min (Worst Case)', linewidth=1.5)
plt.plot(sigma_pct, sndr_mean, color='green', label='Mean', linewidth=2)

```

```
plt.title(r'SNDR vs. Capacitor Mismatch ($\sigma_{\Delta C/C}$')
plt.xlabel(r'$\sigma$ of $\Delta C/C$ (%)')
plt.ylabel('SNDR (dB)')
plt.grid(True, which='both', linestyle='—', alpha=0.5)
plt.legend()
plt.show()

# Parametros-alvo
ENOB_alvo = 10.5 # bits
yield_alvo = 0.99 # 99 % dos chips devem cumprir o requisito
# usamos o percentil 1 (pior 1 %) como criterio

# 1. ENOB a partir dos resultados Monte Carlo existentes
enob_results = (snr_results - 1.76) / 6.02

enob_p1 = np.percentile(enob_results, 1, axis=1) # crit rio yield 99 %
enob_mean = np.mean(enob_results, axis=1)
enob_p99 = np.percentile(enob_results, 99, axis=1)

# 2. _max que satisfaz ENOB_alvo @ yield_alvo
try:
    f_interp = interp1d(enob_p1, SC_mc, kind='linear', fill_value='extrapolate')
    sigma_max = float(f_interp(ENOB_alvo))
    sigma_max = np.clip(sigma_max, SC_mc[0], SC_mc[-1])
except Exception:
    sigma_max = SC_mc[np.argmin(np.abs(enob_p1 - ENOB_alvo))]

sigma_max_pct = sigma_max * 100 # em %

print(f'\n{' '*65}')
print(f"Estimacao para ENOB {ENOB_alvo} bits (yield {yield_alvo*100:.0f}%)")
print(f{' '*65}')
print(f"_max ( C /C) = {sigma_max:.5f} ({sigma_max_pct:.4f}%)")

# 3. area minima de Cu modelo do paper (Fig. 4)
#
# Recta da Fig. 4: (%) = 0.408 / sqrt(Area_um2) + 0.01
#
# Inverter para obter Area:
# sigma_max_pct - 0.01 = 0.408 / sqrt(Area)
# sqrt(Area) = 0.408 / (sigma_max_pct - 0.01)
# Area = (0.408 / (sigma_max_pct - 0.01))^2
#
# Se sigma_max_pct <= 0.01 (abaixo do patamar do modelo), area seria infinita
# nesse caso usa-se o maior valor do eixo x da Fig. 4 (area 16 um^2).

a_fit = 0.408 # coeficiente da recta y = a/sqrt(Area) + b (Fig. 4)
b_fit = 0.01 # offset

delta = sigma_max_pct - b_fit
if delta > 1e-6:
    Area_Cu_um2 = (a_fit / delta) ** 2
else:
    Area_Cu_um2 = (a_fit / (0.112 - b_fit)) ** 2 # fallback: ponto (16, 0.112)
    print(f"AVISO: _max muito baixo; usando Area_Cu = 16 um^2 como limite superior.")

# Lado do condensador quadrado equivalente
lado_Cu = np.sqrt(Area_Cu_um2)

print(f'\nModelo mismatch (Fig. 4): (%) = {a_fit} / Area + {b_fit}')
```

```

print(f"Area minima de Cu = {Area_Cu_um2:.2f} um^2 ({lado_Cu:.2f} {lado_Cu:.2f}

# Cu real do paper para comparacao
Area_Cu_paper = 4 * 4 # um^2
sigma_paper = a_fit / np.sqrt(Area_Cu_paper) + b_fit
print(f"\n[Referencia paper] Cu = 4 4 um^2 {sigma_paper:.3f}%")

# 4. Capacidade de Cu e total do DAC
#
# O paper indica:
# - Cu_paper = 6.5 fF para Area = 16 um^2
# - Densidade: C_ox = 6.5 / 16 = 0.406 fF/um^2
#
# Escalar linearmente para a area estimada.

C_ox_fF_um2 = 6.5 / 16.0 # fF/um^2 (do paper, texto Sec. III-B)
Cu_fF = Area_Cu_um2 * C_ox_fF_um2

# N mero total de unidades Cu numa array single-ended (da Tabela 1 do paper):
# MSB (bits 1-7, com redundancia): 56+32+16+10+6+4+2 = 126 Cu
# C6 (bridge): 2 Cu (CB = 2Cu conforme paper)
# LSB (bits 8-14, com redundancia): 1+7+4+2+1+1+15 = 31 Cu (aprox., ver Tabela 1)
# C_total single-ended paper 840 fF / 6.5 fF 129 Cu usa valor do paper

N_Cu_single = round(840 / 6.5) # 129 unidades Cu por ramo single-ended
N_Cu_total = 2 * N_Cu_single # diferencial (p + n)

C_total_fF = N_Cu_total * Cu_fF

# Escalar a rea total do DAC
Area_DAC_um2 = N_Cu_total * Area_Cu_um2

print(f"\nDensidade C_ox = {C_ox_fF_um2:.3f} fF/ m ")
print(f"Cu estimado = {Cu_fF:.2f} fF")
print(f"N_Cu (single-ended) {N_Cu_single} unidades Cu")
print(f"C_total DAC (diferencial) {C_total_fF:.0f} fF")
print(f"Area total DAC {Area_DAC_um2:.0f} m ({Area_DAC_um2*1e-6:.4f})

# 5. Energia por conversao (DAC)
#
# Para um SAR com switching monotonic/hibrido, a energia media de
# chaveamento do DAC e aproximadamente:
# E_DAC (1/2) C_total Vref^2 (limite superior conservador)
#
# Vref = 1.2 V (paper, Sec. II)
#
# FoM_DAC = E_DAC / 2^ENOB

Vref_paper = 1.2 # V
alpha = 0.5 # factor de chaveamento medio SAR
E_DAC_fJ = alpha * C_total_fF * (Vref_paper ** 2)
FoM_DAC = E_DAC_fJ / (2 ** ENOB_alvo)

# Comparacao com o paper (ENOB=10.17, P=0.8mW, fs=100MHz)
E_total_paper_fJ = (0.8e-3 / 100e6) * 1e15 # fJ por conversao (total)
FoM_paper = E_total_paper_fJ / (2**10.17)

print(f"\nVref = {Vref_paper} V")
print(f"Energia DAC estimada {E_DAC_fJ:.1f} fJ/conversao")
print(f"FoM_DAC estimado {FoM_DAC:.2f} fJ/conversion-step")
print(f"\n[Referencia paper] E_total = {E_total_paper_fJ:.1f} fJ FoM = {FoM_paper}

```

```
print(f"{'='*65}\n")

# 6. Gráfico ENOB vs Capacitor com marcação do ponto-alvo
sigma_pct_plot = SC_mc * 100

fig, ax = plt.subplots(figsize=(10, 6))

for s_idx, s_val in enumerate(sigma_pct_plot):
    ax.scatter(np.ones(Numero_chips) * s_val, enob_results[s_idx, :],
               facecolors='none', edgecolors='tab:blue', alpha=0.2, s=8)

ax.plot(sigma_pct_plot, enob_p99, color='blue', label='Percentil_99_(melhor)',
        linewidth=1.5)
ax.plot(sigma_pct_plot, enob_mean, color='green', label='Média',
        linewidth=2)
ax.plot(sigma_pct_plot, enob_p1, color='red', label='Percentil_1_(critério_yield_99%)')

ax.axhline(ENOB_alvo, color='black', linestyle='—', linewidth=1.2,
           label=f'Alvo: {ENOB_alvo} bits')
ax.axvline(sigma_max_pct, color='darkorange', linestyle=':', linewidth=1.5,
           label=f'_max {sigma_max_pct:.3f}% (yield {yield_alvo*100:.0f}%)')

# Ponto do papel
ax.axvline(sigma_paper, color='purple', linestyle='-.', linewidth=1.2,
           label=f'Paper: {Cu44_m} {sigma_paper:.3f}%')

ax.set_title(r"ENOB vs. Capacitor Mismatch (\sigma_{\Delta C/C}) Alinhado")
ax.set_xlabel(r"\sigma_{de} \Delta C/C$ (%)")
ax.set_ylabel("ENOB (bits)")
ax.legend(fontsize=9)
ax.grid(True, which='both', linestyle='—', alpha=0.5)
plt.tight_layout()
plt.show()
```